

RUSH 01

Guia de implementação em C - Piscine 42

O que muda em relação ao guia anterior: agora cada fase contém código-base ou esqueletos concretos para implementares. O algoritmo completo não é entregue como uma solução pronta; vais montar as peças na ordem certa.

Base: enunciado Rush 01 versão 8 fornecido por ti. O enunciado exige um solver 4x4, compilação com `cc -Wall -Wextra -Werror`, e indica `write`, `malloc` e `free` como funções autorizadas. Também exige `Error + newline` em caso de erro.

Regra: copia apenas os pequenos blocos que já compreendes. Nos pontos marcados para implementação, escreve tu a lógica. No fim de cada fase tens um teste e um checkpoint.

Mapa do projeto

Fase 0	Modelo mental e índices
Fase 1	Header, ficheiros e grid
Fase 2	Parser
Fase 3	Output
Fase 4	Visibility + duplicados
Fase 5	Constraints
Fase 6	Backtracking
Fase 7	Integração
Fase 8	Testes/debug
Fase 9	Norm/defesa
Bónus	5x5 até 9x9

Estrutura sugerida

```
ex00/  
■■■ rush01.h  
■■■ main.c  
■■■ input.c  
■■■ grid.c  
■■■ checks.c  
■■■ solver.c  
■■■ print.c
```

É uma sugestão. Podes fundir ficheiros, mas tenta manter responsabilidades pequenas.

Fase 0 - Modelo mental antes do C

O problema tem uma grelha 4x4, alturas 1..4 sem repetição em cada linha e coluna, e 16 pistas: 4 de cima, 4 de baixo, 4 da esquerda e 4 da direita. O formato das pistas e do output está no enunciado.

0.1 Fixar índices

```
grid[row][col]
row = 0..3
col = 0..3

clues[0..3] = top
clues[4..7] = bottom
clues[8..11] = left
clues[12..15] = right
```

0.2 Posição linear

```
row = pos / 4;
col = pos % 4;
```

A escolha da ordem dos clues não vem imposta como array em C pelo enunciado; este mapa é uma convenção do teu programa. Mantém-na constante em todas as funções.

Checkpoint 0

- [] Consigo apontar para cada uma das 16 clues sem hesitar.
- [] Consigo explicar $\text{pos} / 4$ e $\text{pos} \% 4$.

Fase 1 - Header, ficheiros e grid

1.1 Header base

```
#ifndef RUSH01_H
#define RUSH01_H

#include <unistd.h>

void init_grid(int grid[4][4]);
int parse_input(char *str, int clues[16]);
void print_error(void);
void print_grid(int grid[4][4]);
int visible_count(int line[4]);
int valid_in_row(int grid[4][4], int row, int value);
int valid_in_col(int grid[4][4], int col, int value);
int can_place(int grid[4][4], int row, int col, int value);
int check_complete_grid(int grid[4][4], int clues[16]);
int solve(int grid[4][4], int clues[16], int pos);

#endif
```

Este bloco pode ser copiado para o teu rush01.h. As declarações têm de corresponder exatamente às funções que fores a criar.

1.2 Inicialização da matriz

```
void init_grid(int grid[4][4])
{
    int row;
    int col;

    row = 0;
    while (row < 4)
    {
        col = 0;
        while (col < 4)
        {
            grid[row][col] = 0;
            col++;
        }
        row++;
    }
}
```

Usa 0 para "vazio". Não precisas de memória dinâmica para a versão obrigatória.

1.3 Compilar sempre

```
cc -Wall -Wextra -Werror -o rush-01 *.c
```

Checkpoint 1

- [] O projeto compila sem warnings.
- [] init_grid deixa todas as células a 0.

Fase 2 - Parser: argc/argv -> clues

2.1 main de arranque

```
int main(int argc, char **argv)
{
    int clues[16];
    int grid[4][4];

    if (argc != 2)
    {
        print_error();
        return (1);
    }
    if (!parse_input(argv[1], clues))
    {
        print_error();
        return (1);
    }
    init_grid(grid);
    /* solver entra mais tarde */
    return (0);
}
```

2.2 Ler um clue

```
int value;

if (str[i] < '1' || str[i] > '4')
    return (0);
value = str[i] - '0';
```

2.3 Esqueleto do parser

```
int parse_input(char *str, int clues[16])
{
    int i;
    int count;

    i = 0;
    count = 0;
    while (str[i] != '\0')
    {
        /* Espera um dígito 1..4. */
        /* Guarda-o em clues[count]. */
        /* Avança e trata o espaço. */
        /* Rejeita count > 16. */
    }
    return (count == 16);
}
```

Aqui tens de implementar a parte dos separadores. Para o obrigatório, cada clue é um único dígito. Não aceites letras ou números fora de 1..4.

2.4 Error

```
void print_error(void)
{
    write(1, "Error\n", 6);
}
```

Testes

```
./rush-01
./rush-01 "1 2 3"
./rush-01 "1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 5"
./rush-01 "1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4"
```

Checkpoint 2

[] clues[0]..clues[15] recebem os valores certos.

[] Input inválido nunca entra no solver.

Fase 3 - Output e manipulação da grelha

3.1 print_grid

```
void print_grid(int grid[4][4])
{
    int row;
    int col;
    char c;

    row = 0;
    while (row < 4)
    {
        col = 0;
        while (col < 4)
        {
            c = grid[row][col] + '0';
            write(1, &c, 1);
            if (col < 3)
                write(1, " ", 1);
            col++;
        }
        write(1, "\n", 1);
        row++;
    }
}
```

O output obrigatório é uma matriz 4x4 com números separados por espaços e newline em cada linha.

3.2 Testar sem solver

```
grid[0][0] = 1;
grid[0][1] = 2;
grid[0][2] = 3;
grid[0][3] = 4;
print_grid(grid);
```

Checkpoint 3

- [] Consigo imprimir qualquer grid.
- [] Não há lógica de clues dentro de print_grid.

Fase 4 - Visibility e regras locais

4.1 Contar torres visíveis

```
int visible_count(int line[4])
{
    int i;
    int max;
    int count;

    i = 0;
    max = 0;
    count = 0;
    while (i < 4)
    {
        if (line[i] > max)
        {
            max = line[i];
            count++;
        }
        i++;
    }
    return (count);
}
```

Exemplo: 2 1 4 3 produz 2 visíveis. O 2 aparece primeiro, o 1 fica escondido, o 4 torna-se novo máximo, o 3 fica escondido.

4.2 Validar linha

```
int valid_in_row(int grid[4][4], int row, int value)
{
    int col;

    col = 0;
    while (col < 4)
    {
        if (grid[row][col] == value)
            return (0);
        col++;
    }
    return (1);
}
```

4.3 Validar coluna

```
int valid_in_col(int grid[4][4], int col, int value)
{
    int row;

    row = 0;
    while (row < 4)
    {
        if (grid[row][col] == value)
            return (0);
        row++;
    }
    return (1);
}
```

4.4 can_place

```
int can_place(int grid[4][4], int row, int col, int value)
{
    if (!valid_in_row(grid, row, value))
        return (0);
    if (!valid_in_col(grid, col, value))
        return (0);
    return (1);
}
```

4.5 Direções

```
left = line[0], line[1], line[2], line[3]
right = line[3], line[2], line[1], line[0]
top = grid[0][col], grid[1][col], grid[2][col], grid[3][col]
bottom= grid[3][col], grid[2][col], grid[1][col], grid[0][col]
```

Não precisas de quatro funções de contagem. Usa a mesma `visible_count()` numa ordem diferente.

Checkpoint 4

- [] `visible_count` funciona de forma independente.
- [] `can_place` rejeita duplicados na linha/coluna.
- [] Sei construir as quatro sequências de leitura.

Fase 5 - Constraints: ligar clues ao grid

5.1 Validar uma linha completa

```
/* Depois de completa */
left_count = ...;
right_count = ...;

if (left_count != clues[8 + row])
return (0);
if (right_count != clues[12 + row])
return (0);
```

Os ... significam que tens de construir os arrays temporários e chamar `visible_count()`. Não copies literalmente os três pontos.

5.2 Validar uma coluna completa

```
/* Coluna col */
/* construir top_to_bottom[] */
/* construir bottom_to_top[] */
/* comparar com clues[col] e clues[4 + col] */
```

5.3 Função final

```
int check_complete_grid(int grid[4][4], int clues[16])
{
/* 1. Verifica linhas completas. */
/* 2. Verifica colunas completas. */
/* 3. Retorna 0 na primeira clue que falhar. */
/* 4. Se todas passarem, retorna 1. */
}
```

5.4 Pruning mínimo

Começa a verificar clues apenas quando a linha ou coluna está completa. Isto é mais lento do que poda parcial, mas é muito mais fácil de escrever e depurar. Depois de funcionar, acrescenta melhorias: clue 1 -> extremo é 4; clue 4 -> sequência vista é 1,2,3,4.

Checkpoint 5

- [] Uma grelha completa é aceite só se respeitar todas as clues.
- [] Sei explicar o mapa clues[0..15].
- [] Ainda não preciso de heurísticas avançadas para ter um solver correto.

Fase 6 - Backtracking

O objetivo é preencher uma célula, avançar, e voltar atrás quando um ramo deixa de poder produzir solução. A parte essencial é o estado de entrada e o desfazer.

6.1 Esqueleto do solve

```
int solve(int grid[4][4], int clues[16], int pos)
{
    int row;
    int col;
    int value;

    if (pos == 16)
        return (check_complete_grid(grid, clues));
    row = pos / 4;
    col = pos % 4;
    value = 1;
    while (value <= 4)
    {
        /* Se pode colocar value */
        /* Colocar value */
        /* Tentar solve(pos + 1) */
        /* Se falhar, grid[row][col] volta a 0 */
        value++;
    }
    return (0);
}
```

6.2 Implementar por micro-passos

```
if (can_place(grid, row, col, value))
{
    grid[row][col] = value;
    /* próximo passo: chamar solve */
    grid[row][col] = 0;
}
```

Primeiro testa este comportamento sem recursão: verifica que o valor entra e sai. Depois introduz a chamada recursiva.

6.3 Introduzir recursão

```
grid[row][col] = value;
if (solve(grid, clues, pos + 1))
    return (1);
grid[row][col] = 0;
```

O return (1) é o sinal de "encontrei uma solução em baixo; não procures outra". Se a chamada recursiva devolver 0, fazes backtrack e tentas o próximo valor.

6.4 Pruning por linha/coluna

```
/* depois de colocar */
if (col == 3)
{
    /* a linha ficou completa -> verificar left/right */
}
if (row == 3)
{
    /* a coluna ficou completa -> verificar top/bottom */
}
```

Checkpoint 6

- [] Cada tentativa falhada volta a colocar a célula em 0.
- [] A primeira solução faz o solve propagar 1 até ao main.
- [] Se nenhum value funcionar, a função devolve 0.

Fase 7 - Integração final

7.1 Main final - esqueleto

```
int main(int argc, char **argv)
{
    int grid[4][4];
    int clues[16];

    if (argc != 2)
    {
        print_error();
        return (1);
    }
    if (!parse_input(argv[1], clues))
    {
        print_error();
        return (1);
    }
    init_grid(grid);
    if (solve(grid, clues, 0))
        print_grid(grid);
    else
        print_error();
    return (0);
}
```

7.2 Teste oficial

```
./rush-01 "4 3 2 1 1 2 2 2 4 3 2 1 1 2 2 2"
```

O enunciado fornece como saída de referência: 1 2 3 4 / 2 3 4 1 / 3 4 1 2 / 4 1 2 3.

7.3 Não confundir dois tipos de erro

Input inválido: falha no parser. **Puzzle sem solução:** parser aceita, mas `solve()` devolve 0. O output externo é o mesmo: Error + newline.

Checkpoint 7

- [] O exemplo do enunciado produz uma solução válida.
- [] Um input impossível produz Error.
- [] Um input inválido produz Error.

Fase 8 - Testes e debugging

8.1 Testar visibility isoladamente

```
1 2 3 4 -> 4
4 3 2 1 -> 1
2 1 4 3 -> 2
1 3 2 4 -> 3
```

8.2 Testar índices

Cria uma grelha de teste em que cada célula é fácil de identificar e confirma que top, bottom, left e right usam a ordem certa. Um erro aqui pode fazer um solver perfeitamente correto parecer errado.

8.3 Debug do backtracking

```
/* temporariamente */
/* imprime pos, row, col e value */
/* remove antes da entrega */
```

8.4 Checklist de inputs

- ☐ argc = 1
- ☐ argc > 2
- ☐ menos de 16 clues
- ☐ mais de 16 clues
- ☐ 0
- ☐ 5
- ☐ letras
- ☐ espaços fora do formato
- ☐ input válido
- ☐ puzzle sem solução

8.5 Se travar

Parser: mostra os 16 clues. **Direções:** mostra as quatro sequências. **Visibility:** testa fora do solver. **Backtracking:** mostra quando uma célula é preenchida e quando volta a 0.

Checkpoint 8

- ☐ Cada bug tem um teste que o reproduz.
- ☐ Todos os prints de debug foram removidos.

Fase 9 - Norm e defesa

O enunciado destaca a Norm, as flags de compilação e a necessidade de todos os elementos da equipa conhecerem todo o projeto para a avaliação.

- ☐ Compila com `cc -Wall -Wextra -Werror -o rush-01 *.c`.
- ☐ Nenhuma função está desnecessariamente gigante.
- ☐ Não há código morto ou debug.
- ☐ Nomes de variáveis são explicáveis.
- ☐ Cada ficheiro tem uma responsabilidade clara.
- ☐ Só dependes das funções autorizadas.

Perguntas da defesa

- ☐ Por que `argc` é 2?
- ☐ O que contém `argv[1]`?
- ☐ Por que `argv` é `char **`?
- ☐ Como transformas o carácter 3 em inteiro 3?
- ☐ Como `pos` vira `row/col`?
- ☐ Como funciona `visible_count`?
- ☐ Por que uma `clue 1` força um 4 no extremo?
- ☐ Por que `can_place` verifica linha e coluna?
- ☐ O que é `backtracking`?
- ☐ Por que voltas a pôr a célula em 0?
- ☐ Por que `solve` pode retornar 1 imediatamente?

Teste final: escolhe qualquer função do projeto ao acaso e explica linha a linha sem consultar o PDF. Se não conseguires, ainda há uma parte que precisas de compreender.

BÔNUS - Generalizar até 9x9

Só fazes isto depois do 4x4 estar sólido. O enunciado permite mapas maiores até 9x9 e avisa que uma versão obrigatória com falhas não é salva pelo bônus.

B.1 Tamanho

```
int size;
int grid[9][9];
int clues[36];
```

Substitui loops com 4 por loops com size. Valores válidos passam a 1..size.

B.2 Parser multi-dígito

```
value = 0;
while (str[i] >= '0' && str[i] <= '9')
{
    value = value * 10 + (str[i] - '0');
    i++;
}
```

Depois valida $1 \leq \text{value} \leq \text{size}$ e lida com separadores.

B.3 Visibility genérica

```
i = 0;
while (i < size)
{
    /* mesma lógica de max/count */
    i++;
}
```

B.4 Onde o solver começa a sofrer

O número de permutações cresce muito rapidamente. Podes evoluir para candidatos por linha, pré-calcular padrões de visibilidade ou escolher posições com mais restrições. O objetivo é reduzir estados explorados, não apenas acrescentar código.

B.5 Ordem recomendada

- ☐ Parametrizar size.
- ☐ Generalizar parser.
- ☐ Generalizar print.
- ☐ Generalizar duplicate checks.
- ☐ Generalizar visibility.
- ☐ Testar 5x5.
- ☐ Medir estados explorados.
- ☐ Só depois otimizar.

Checklist final

- ☐ Ficheiros corretos na ex00/.
- ☐ Compilação exata do enunciado.
- ☐ Parser robusto.
- ☐ Visibility correta.
- ☐ Linhas/colunas sem repetição.
- ☐ Todas as clues respeitadas.
- ☐ Error exato em casos de erro.
- ☐ Sem debug.
- ☐ Norm verificada.
- ☐ Defesa preparada.
- ☐ Bónus só depois do obrigatório.

A sequência que deves seguir é: parser -> grid -> print -> visibility -> duplicate checks -> clue checks -> backtracking -> integração -> testes -> Norm -> bónus.